

Exercício (Adaptado): Petshop no Boilerplate — "Meus Agendamentos"

Turma: LionsDev

Tópicos: Boilerplate, rotas protegidas com JWT, dono do recurso via token, Mongoose com `ObjectId` / `ref`, CRUD por dono, regra de negócio no service, validações e status codes.

1. Contexto

No Módulo 08 você criou a API do **PetLions** com tudo dentro do `server.js`. Agora vamos **reaproveitar a mesma ideia**, mas dentro do **boilerplate em camadas** e com uma diferença importante: cada agendamento passa a **pertencer a um usuário logado**.

Pense num app onde cada tutor faz login e gerencia **somente os agendamentos dele**. Ninguém vê nem mexe nos agendamentos de outra pessoa.

Este é o mesmo padrão do recurso **Livro** que montamos em aula: um recurso amarrado ao dono (`req.usuario.id`).

2. Ponto de Partida: o Boilerplate

Partimos do **boilerplate LionsDev** (o mesmo da aula):

<https://github.com/nicolassmotta/boilerplate-lions-dev.git>

1. Clone o boilerplate e rode `npm install`.
2. Crie o `.env` a partir do `.env.example` e preencha `MONGO_URI`, `JWT_SECRET`, `JWT_EXPIRES_IN` e `BCRYPT_SALT_ROUNDS`.
3. Suba o servidor e confirme `GET /` respondendo.

O boilerplate **já vem pronto** com:

- A camada de **Usuario** completa (model, repository, service, controller, rotas).
- Cadastro e login com **bcrypt** e **JWT**: `POST /api/auth/cadastro` e `POST /api/auth/login`.
- O middleware **autenticar** (`src/middlewares/autenticacao.middleware.js`), que valida o token e preenche `req.usuario = { id, email }`.

- O helper `criarErro` e o middleware central de erro.

Você **não vai mexer** na autenticação. Vai apenas **criar o recurso** `Agendamento` e amarrá-lo ao **dono**.

Antes de começar, pegue seu token: faça `POST /api/auth/cadastro` (ou `/login`), copie o `token` da resposta e envie em **todas** as rotas novas no header `Authorization: Bearer SEU_TOKEN`.

3. Regras de Ouro (valem para todas as rotas do recurso)

1. **Toda rota é protegida.** Aplique `router.use(authenticar)` no topo do arquivo de rotas.
2. **O dono vem sempre do token** (`req.usuario.id`), **nunca do body**. Mesmo que mandem `usuario` no body, ignore.
3. **Toda consulta filtra pelo dono.** Listar, buscar, atualizar e remover só mexem nos agendamentos daquele usuário.
4. **Filtre pelo dono já na consulta:** `{ _id: idAgendamento, usuario: idDoUsuario }`. Se não achar, é `404` — sem `if` de autorização. "Não existe" e "não é seu" têm a mesma resposta, de propósito.
5. **Listagem nunca dá 404:** se o usuário não tem agendamentos, devolva um array vazio.

4. Arquivos que você vai criar

Camada	Arquivo	Use como referência (Usuario)
Model	<code>src/models/agendamento.model.js</code>	<code>usuario.model.js</code>
Repository	<code>src/repositories/agendamento.repository.js</code>	<code>usuario.repository.js</code>
Service	<code>src/services/agendamento.service.js</code>	<code>usuario.service.js</code>
Controller	<code>src/controllers/agendamento.controller.js</code>	<code>usuario.controller.js</code>
Routes	<code>src/routes/agendamento.routes.js</code>	<code>usuario.routes.js</code>

5. Etapa 1 — Model `Agendamento`

Crie `src/models/agendamento.model.js`. Campos:

- `nomePet` : `String`, obrigatório, `trim`.
- `especie` : `String`, obrigatório, `enum`: ["Cão", "Gato", "Outro"] .
- `nomeDono` : `String`, obrigatório, `trim`.
- `telefoneDono` : `String`, obrigatório.
- `servico` : `String`, obrigatório, `enum`: ["Banho", "Tosa", "Banho e Tosa"] .
- `data` : `String`, obrigatório (ex.: "2026-06-15").
- `valor` : `Number` (a **API calcula**, não vem do body).
- `status` : `String`, `enum`: ["Agendado", "Concluído", "Cancelado"], `default`: "Agendado" .
- `usuario` : `ObjectId`, `ref`: "Usuario", **obrigatório** (o dono).
- Ative `timestamps: true` .

```
usuario: {
  type: mongoose.Schema.Types.ObjectId,
  ref: "Usuario",
  required: [true, "O dono do agendamento é obrigatório."],
},
```

Conceito novo — o campo de dono (`ObjectId` + `ref`): o bloco acima é o que amarra o agendamento ao usuário logado. `type: ObjectId` + `ref: "Usuario"` significa "este campo guarda o `_id` de um documento da coleção `Usuario`". É assim que o recurso ganha um dono — e depois você consegue filtrar tudo por ele.

Dica: monte o Schema do zero. Se quiser relembrar a sintaxe, use o `usuario.model.js` do boilerplate como referência. Aqui **não** precisa de `select: false` nem `transform` .

CrITÉRIOS de aceite: `nomePet`, `especie`, `servico`, `data` e `usuario` são obrigatórios; `especie`, `servico` e `status` só aceitam os valores das listas.

6. Etapa 2 — Repository

Crie `src/repositories/agendamento.repository.js`. Cada função conversa com o Model (use o `usuario.repository.js` do boilerplate como referência do formato):

- `criar(dados)` → `Agendamento.create(dados)` .
- `listarPorUsuario(idUsuario)` → `Agendamento.find({ usuario: idUsuario }).sort({ createdAt: -1 })` .

- `buscarPorIdDoDono(idAgendamento, idUsuario) → Agendamento.findOne({ _id: idAgendamento, usuario: idUsuario })` .
- `atualizarPorIdDoDono(idAgendamento, idUsuario, dados) → findOneAndUpdate(filtro, dados, { new: true, runValidators: true })` .
- `deletarPorIdDoDono(idAgendamento, idUsuario) → findOneAndDelete(filtro)` .
- Exporte tudo no objeto `AgendamentoRepository` .

Critérios de aceite: toda consulta filtra por `usuario` ; funções retornam `null` quando não encontram.

7. Etapa 3 — Service (regras de negócio)

Crie `src/services/agendamento.service.js` . Aqui mora a **regra do valor automático** (igual à do Módulo 08):

Espécie	Banho	Tosa	Banho e Tosa
Cão	50	60	100
Gato	60	70	110
Outro	40	50	80

Como transformar a tabela em código (a "regra de valor", calculada no service): em vez de uma pilha de `if` , use um objeto de preços e leia pela chave `especie` → `servico` :

```
const TABELA = {
  "Cão": { "Banho": 50, "Tosa": 60, "Banho e Tosa": 100 },
  "Gato": { "Banho": 60, "Tosa": 70, "Banho e Tosa": 110 },
  "Outro": { "Banho": 40, "Tosa": 50, "Banho e Tosa": 80 },
};
const valor = TABELA[especie]?.[servico];
if (valor === undefined) throw criarErro("Espécie ou serviço inválido.", 400);
```

Depois é só montar o objeto com `valor` calculado e `usuario: idDoUsuario` e mandar pro repository.

Funções:

- `registrar(idDoUsuario, dados)` : calcula o `valor` a partir de `especie` + `servico` , monta o objeto com `usuario: idDoUsuario` e chama `AgendamentoRepository.criar(...)` .
- `listarMeus(idDoUsuario)` : retorna a lista do usuário.
- `buscarMeu(idDoUsuario, idAgendamento)` : chama `buscarPorIdDoDono(...)` . Se vier `null` , lança `criarErro("Agendamento não encontrado.", 404)` .
- `atualizarMeu(idDoUsuario, idAgendamento, dados)` : chama `atualizarPorIdDoDono(...)` . Se vier `null` , lança `404` .
- `removerMeu(idDoUsuario, idAgendamento)` : chama `deletarPorIdDoDono(...)` . Se `null` , lança `404` ; se ok, retorna `{ message: "Agendamento removido com sucesso." }` .

O service **não conhece** `req / res` . Recebe ids e dados, devolve dados ou lança erro.

Critérios de aceite: Cão + Banho grava `valor` 50; Gato + Banho e Tosa grava 110; o dono salvo é sempre o `idDoUsuario` recebido.

8. Etapa 4 — Controller

Crie `src/controllers/agendamento.controller.js` (use o `usuario.controller.js` como referência do padrão, com `try/catch` e `next(error)`):

- `registrar(req, res, next)` → usa `req.usuario.id` e `req.body` ; responde `201` com o agendamento.
- `listarMeus(req, res, next)` → usa `req.usuario.id` ; responde `200` com `{ agendamentos }` .
- `buscarMeu(req, res, next)` → usa `req.usuario.id` e `req.params.id` ; responde `200` .
- `atualizarMeu(req, res, next)` → usa `req.usuario.id` , `req.params.id` , `req.body` ; responde `200` .
- `removerMeu(req, res, next)` → usa `req.usuario.id` e `req.params.id` ; responde `200` .

9. Etapa 5 — Routes e registro no `app.js`

Crie `src/routes/agendamento.routes.js` :

- Importe `Router` , o `AgendamentoController` e o middleware `autenticar` .

- No topo: `router.use(authenticar)` (todas as rotas exigem login).
- Rotas:
 - `router.post("/", AgendamentoController.registrar)`
 - `router.get("/", AgendamentoController.listarMeus)`
 - `router.get("/:id", AgendamentoController.buscarMeu)`
 - `router.patch("/:id", AgendamentoController.atualizarMeu)`
 - `router.delete("/:id", AgendamentoController.removerMeu)`

No `src/app.js`, registre **antes** do middleware de rota não encontrada (404):

```
import agendamentoRoutes from "../routes/agendamento.routes.js";
app.use("/api/agendamentos", agendamentoRoutes);
```

Critérios de aceite: chamar `/api/agendamentos/...` sem token → **401**; a aplicação sobe sem erros.

10. Rotas finais

Método	Rota	Proteção	Descrição
POST	<code>/api/agendamentos</code>	JWT	Criar agendamento (dono = você)
GET	<code>/api/agendamentos</code>	JWT	Listar meus agendamentos
GET	<code>/api/agendamentos/:id</code>	JWT	Ver um agendamento meu
PATCH	<code>/api/agendamentos/:id</code>	JWT	Atualizar um agendamento meu
DELETE	<code>/api/agendamentos/:id</code>	JWT	Remover um agendamento meu

11. Fluxo de Testes (use Postman)

1. Cadastro/login → copie o **token**.
2. **POST** `/api/agendamentos` com um **Cão** + **Banho** → confira **valor** = 50 e **status** = "Agendado".
3. **POST** de novo com um **Gato** + **Banho e Tosa** → confira **valor** = 110.
4. **GET** `/api/agendamentos` → lista só os seus.
5. **GET** `/api/agendamentos/:id` com um id válido → **200**.
6. **PATCH** `/api/agendamentos/:id` com `{ "status": "Concluído" }` → **200**.
7. **DELETE** `/api/agendamentos/:id` → **200**; repita a chamada → **404**.

8. Faça **login com um segundo usuário** e tente `GET /api/agendamentos/:id` de um agendamento do primeiro → deve dar `404`.
 9. Qualquer rota **sem token** → `401`.
-

12. Desafios Bônus

1. `GET /api/agendamentos/busca?nome=fred` — filtra **entre os seus** agendamentos pelo nome do pet (use Query Params + filtro por `usuario`).
 2. `GET /api/agendamentos/resumo` — retorne, em JavaScript, a contagem de agendamentos por `status` e a soma de `valor` apenas dos seus.
 3. Impeça `PATCH` que tente mudar o `usuario` (dono) do agendamento.
-

LionsDev • Professor Nicolas Cardoso Motta

Adaptação do Petshop para o Boilerplate (Auth + camadas) - Módulo 09